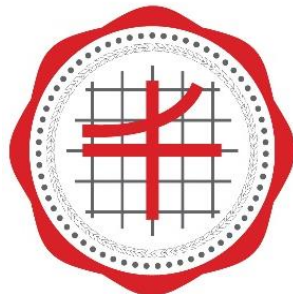


# CPE 121 Mobile Application Developments

---

## บทที่ 1

### การโปรแกรมภาษาจาวา (Introduction to JAVA Programming)



**Faculty of Engineering**

SRINAKHARINWIROT UNIVERSITY

---

**Pramual Choorat, Ph.D.**

**Department of Computer Engineering**

# Java คืออะไร

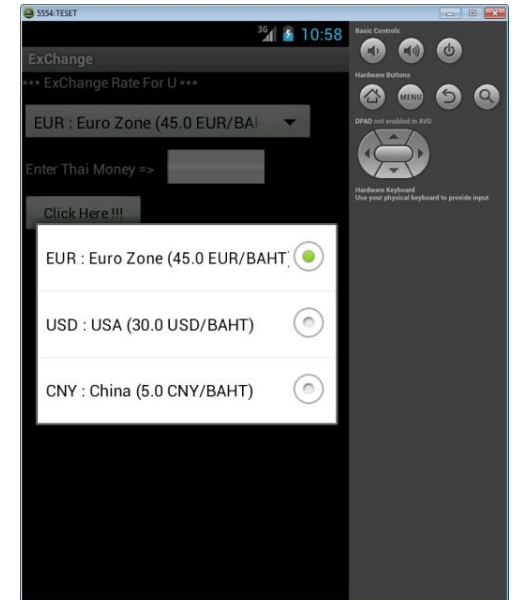
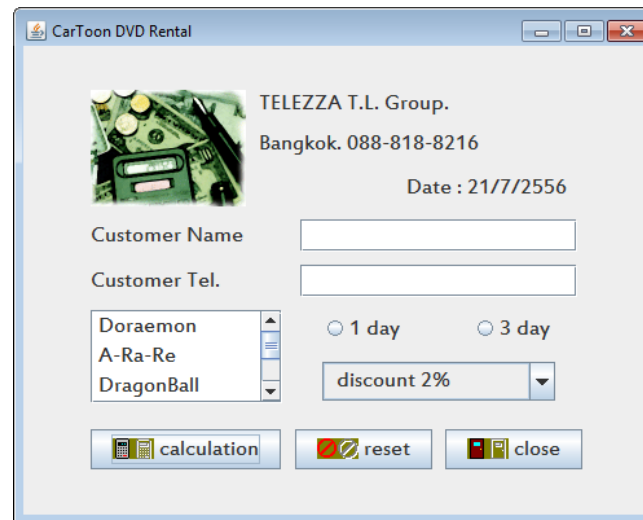
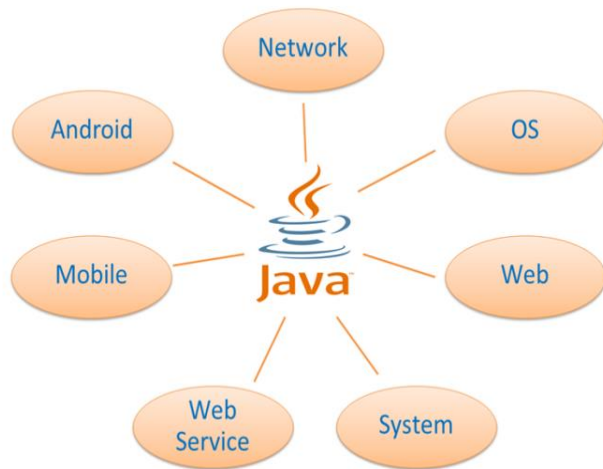
---

- ภาษาโปรแกรมที่พัฒนาขึ้นโดยทีมวิจัยของบริษัท Sun Microsystems
- ใช้สำหรับการเขียนโปรแกรมเชิงวัตถุ
- ง่ายต่อการเรียนและเข้าใจ เนื่องจากมีกลไกของภาษาไม่ซับซ้อน
- ทำงานบนเครื่องคอมพิวเตอร์ที่มีสภาพแวดล้อมต่างกัน ได้โดยไม่ต้องคอมไพล์ใหม่ (Platform Independent)



# Java คืออะไร

- เรียกการทำงานของภาษา Java บนเครื่องคอมพิวเตอร์ที่มีสภาพแวดล้อมต่างกันว่า “จาวาแพลตฟอร์ม” (Java Platform)

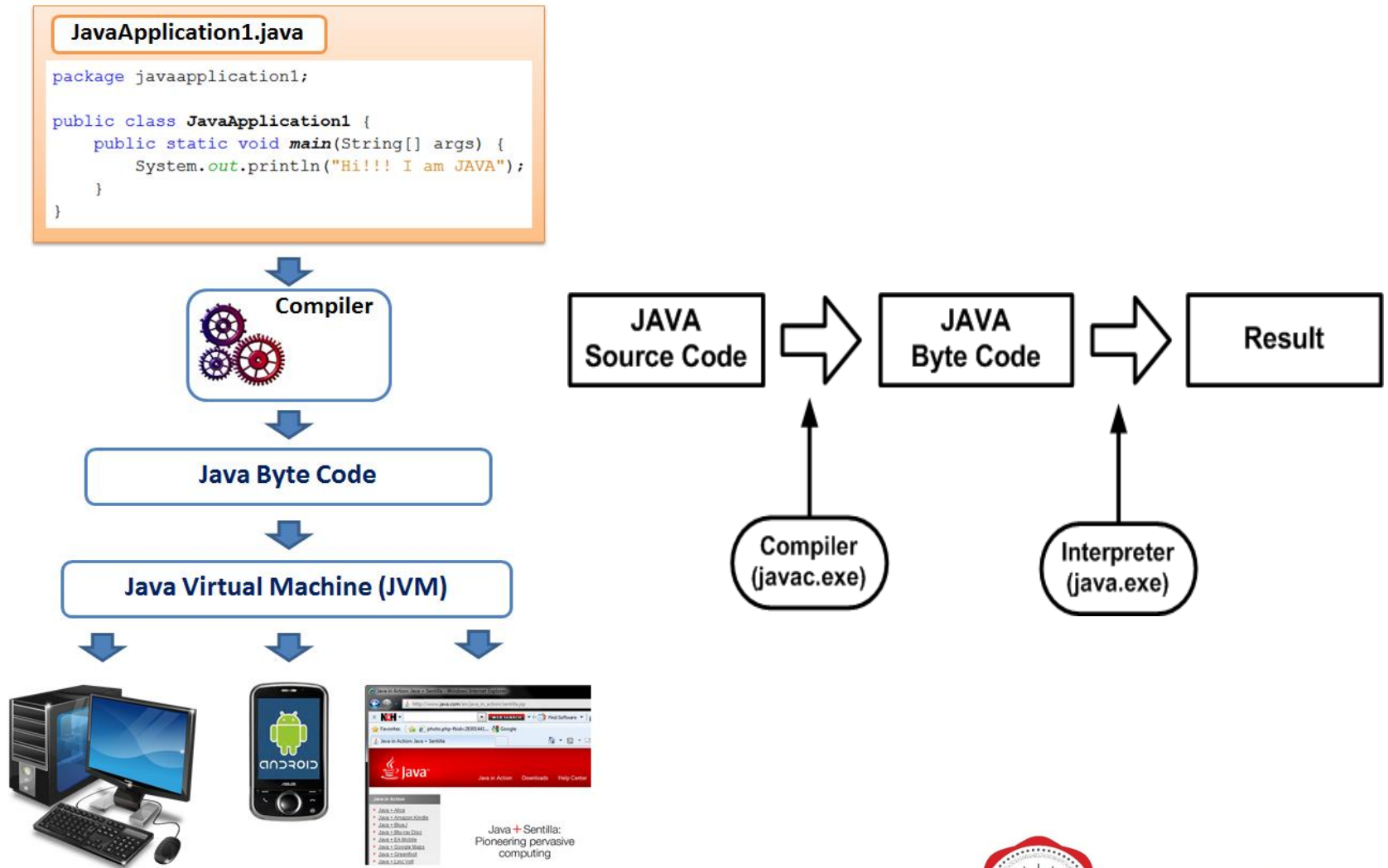


# ชุดเครื่องมือภาษา Java สำหรับจาวาแพลตฟอร์ม

- Java 2 Platform, Standard Edition (J2SE)
  - จาวาแอปพลิเคชัน (Java application)
  - จาวาแอปเพลต (Java applet)
- Java 2 Platform, Enterprise Edition (J2EE)
  - โปรแกรมแบบมัลติเทียร์ (multitiered) สำหรับการพัฒนาโปรแกรมในระดับองค์กร
- Java 2 Platform, Micro Edition (J2ME)
  - สินค้าอิเล็กทรอนิกส์ เช่น โทรศัพท์มือถือ ฯลฯ



# การทำงานของโปรแกรมภาษา Java



# ชุดพัฒนาภาษา Java - J2SE

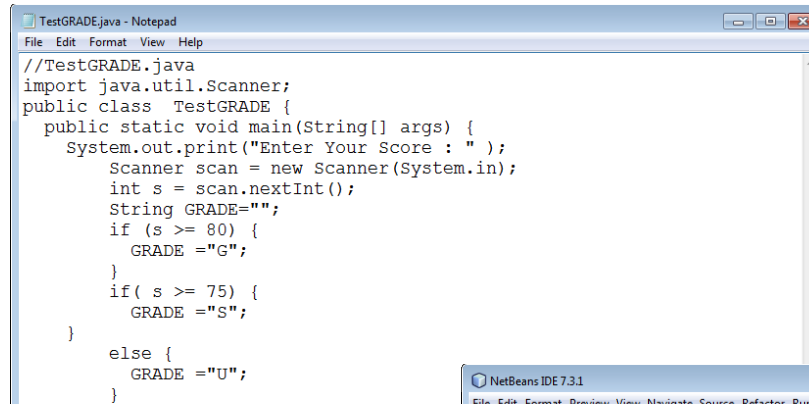
---

- Java Development Kit
  - จาวาคอมไพเลอร์ (javac.exe)
  - สภาพแวดล้อมการรันโปรแกรมจาวา (Java Runtime Environment - JRE) (java.exe)
- Download <http://java.sun.com>
  - Java Development Kit => jdk-7u25-windows-i586.exe
  - Netbeans => netbeans-7.3.1-windows.exe
  - Java Development Kit + Netbeans  
=> jdk-7u25-nb-7\_3\_1-windows-i586.exe

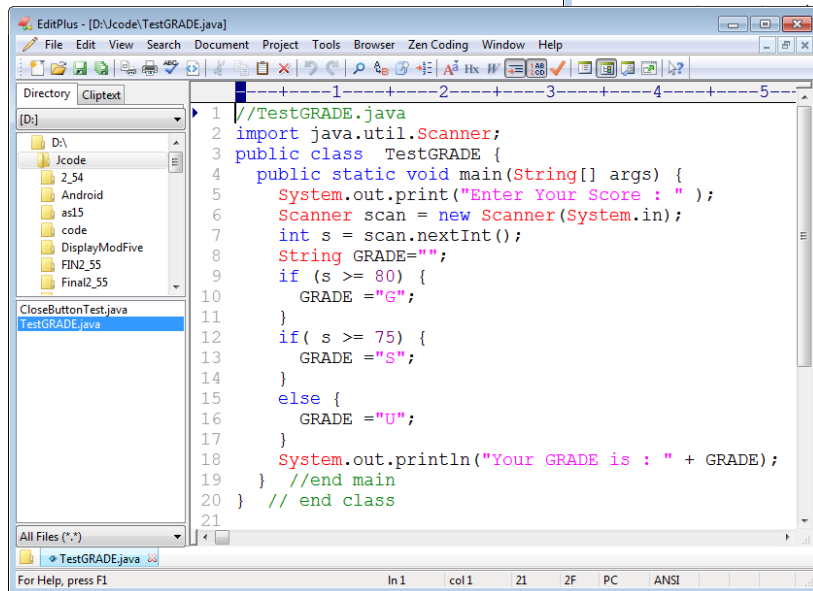


# เครื่องมือพัฒนาโปรแกรมด้วยภาษา Java

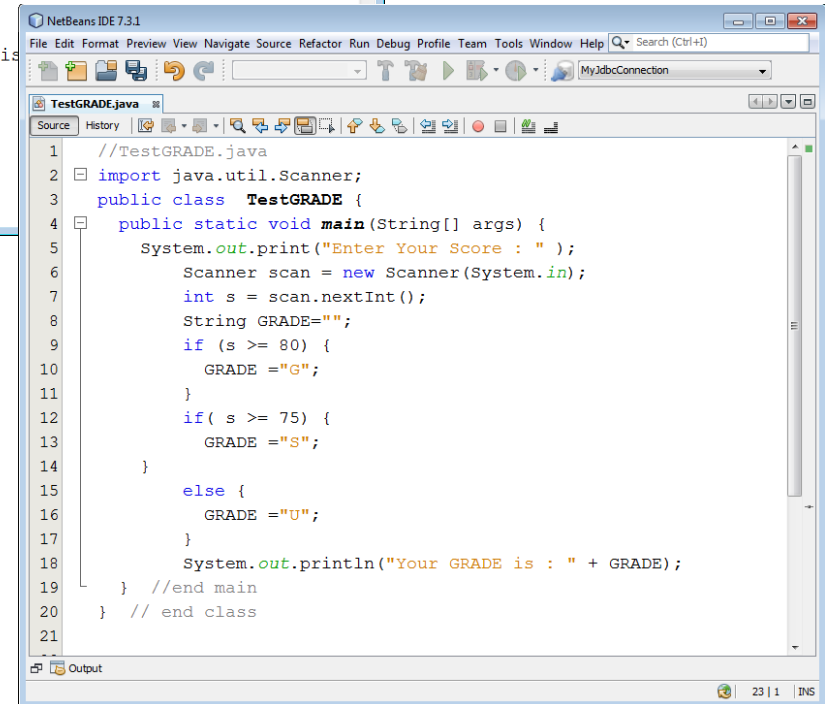
- Notepad
- EditPlus
- NetBeans



```
//TestGRADE.java
import java.util.Scanner;
public class TestGRADE {
    public static void main(String[] args) {
        System.out.print("Enter Your Score : " );
        Scanner scan = new Scanner(System.in);
        int s = scan.nextInt();
        String GRADE="";
        if (s >= 80) {
            GRADE ="G";
        }
        if( s >= 75) {
            GRADE ="S";
        }
        else {
            GRADE ="U";
        }
    }
}
```

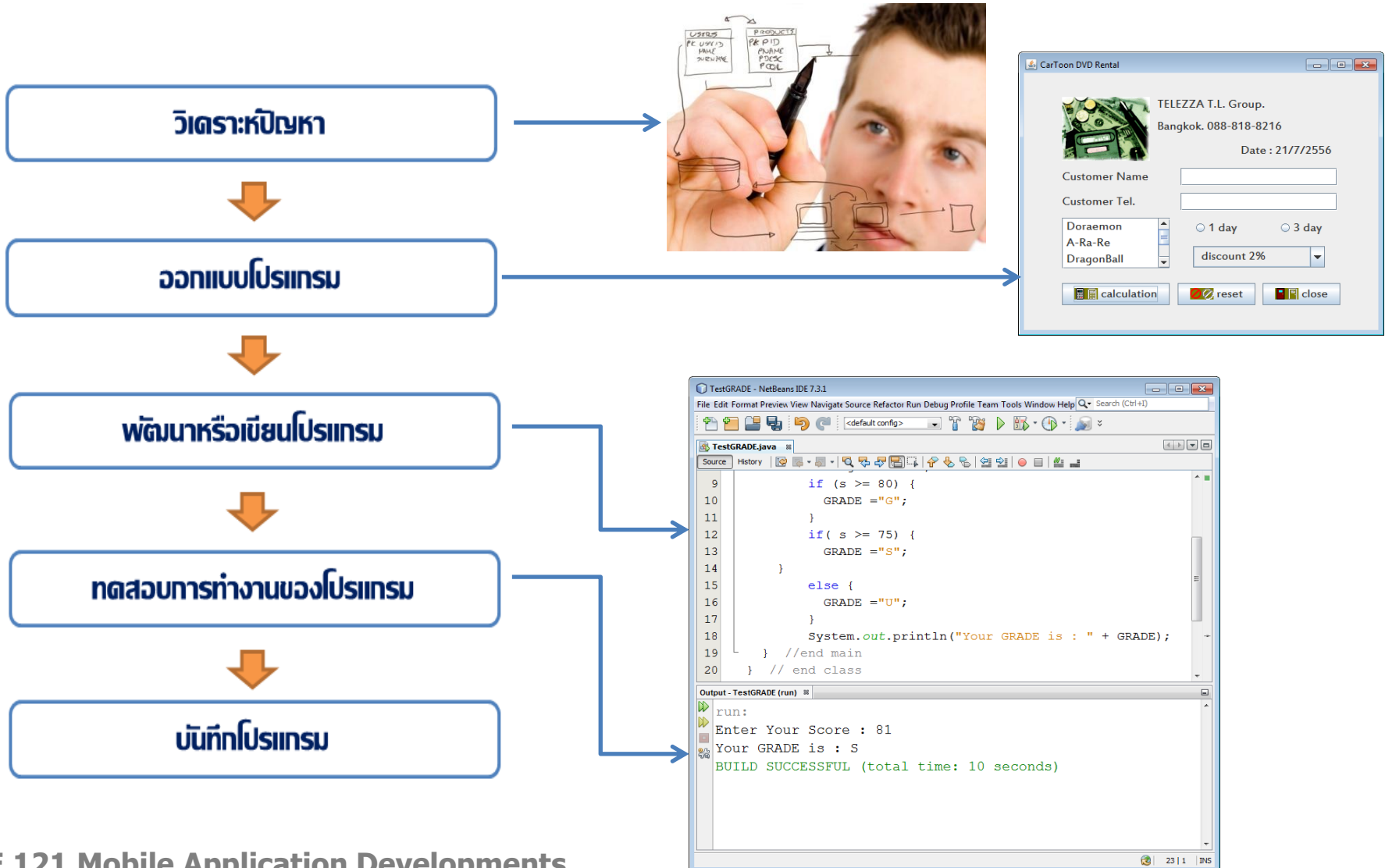


```
1 //TestGRADE.java
2 import java.util.Scanner;
3 public class TestGRADE {
4     public static void main(String[] args) {
5         System.out.print("Enter Your Score : " );
6         Scanner scan = new Scanner(System.in);
7         int s = scan.nextInt();
8         String GRADE="";
9         if (s >= 80) {
10             GRADE ="G";
11         }
12         if( s >= 75) {
13             GRADE ="S";
14         }
15         else {
16             GRADE ="U";
17         }
18         System.out.println("Your GRADE is : " + GRADE);
19     } //end main
20 } // end class
21
```



```
1 //TestGRADE.java
2 import java.util.Scanner;
3 public class TestGRADE {
4     public static void main(String[] args) {
5         System.out.print("Enter Your Score : " );
6         Scanner scan = new Scanner(System.in);
7         int s = scan.nextInt();
8         String GRADE="";
9         if (s >= 80) {
10             GRADE ="G";
11         }
12         if( s >= 75) {
13             GRADE ="S";
14         }
15         else {
16             GRADE ="U";
17         }
18         System.out.println("Your GRADE is : " + GRADE);
19     } //end main
20 } // end class
21
```

# ขั้นตอนการเขียนโปรแกรมภาษา Java





# รูปแบบการเขียนโปรแกรมภาษา Java

```
public class JavaApplication1 {  
    public static void main(String[] args) {  
        System.out.println("Hi!!! I am JAVA");  
    }  
}
```

- คลาสชื่อ **javaApplication1** ชื่อไฟล์จึงต้องเป็น **javaApplication1.java**
- เมื่อโปรแกรมเริ่มทำงาน JRE จะค้นหาเมธอด **main()** เพื่อประมวลผลตามชุดคำสั่งที่เขียนไว้ภายในเครื่องหมายวงเล็บ { }
- เมธอด **main()** จะต้องมียุคิยเวิร์ด **public, static, void** และพารามิเตอร์ที่เป็นอาร์เรย์ของข้อมูลประเภทข้อความ (**String**)
- ทุกคำสั่งต้องจบด้วยเครื่องหมาย ; เสมอ



# คำอธิบายในโปรแกรม

- คำอธิบายบรรทัดเดียวใช้เครื่องหมาย //

```
public class JavaApplication1 {  
    public static void main(String[] args) {  
        // This is my first program  
        System.out.println("Hi!!! I am JAVA");  
    }  
}
```

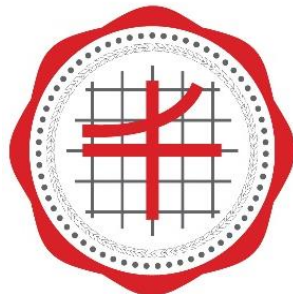
- คำอธิบายที่เป็นย่อหน้า ใช้เครื่องหมาย /\* \*/

```
public class JavaApplication1 {  
    public static void main(String[] args) {  
        /* This is my first program */  
        System.out.println("Hi!!! I am JAVA");  
    }  
}
```

# CPE 121 Mobile Application Developments

---

## การเขียนโปรแกรมเชิงวัตถุ (Introduction to Object Oriented Programming)



**Faculty of Engineering**

SRINAKHARINWIROT UNIVERSITY

---

**Pramual Choorat, Ph.D.**

**Department of Computer Engineering**

# โปรแกรมเชิงวัตถุ

การเขียนโปรแกรมเชิงวัตถุหรือการเขียนโปรแกรมแบบออบเจกต์จะมองทุกอย่างของปัญหาเป็นวัตถุหรือออบเจกต์ (Object) ถ้าหากวัตถุใดมีลักษณะคล้ายกันก็จะรวมทั้งหมดให้เป็นคลาส (Class) ถ้าหากมีการประกาศตัวแปรหรือสร้างข้อมูลขึ้นมา ข้อมูลนั้นก็จะถูกใช้ในออบเจกต์นั้นๆ การกระทำของออบเจกต์จะกระทำผ่านเมธอดของคลาสนั้นๆ โดยมีข้อแตกต่างจากโปรแกรมแบบมีโครงสร้างดังนี้

| ภาษาแบบมีโครงสร้าง                                                                                                                                                   | ภาษาเชิงวัตถุ                                                                                                                                                                                       |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>▪ <b>Top-down Design</b></li><li>▪ เน้นขั้นตอนกระบวนการ</li><li>▪ ใช้ข้อมูลร่วมกัน</li><li>▪ แยกข้อมูลออกจากฟังก์ชัน</li></ul> | <ul style="list-style-type: none"><li>▪ <b>Bottom-up Design</b></li><li>▪ เน้นการมองเป็นวัตถุ</li><li>▪ ซ่อนรายละเอียดข้อมูล (Information hiding)</li><li>▪ ห่อหุ้มข้อมูล (Encapsulation)</li></ul> |

# คลาส ออบเจกต์ แอตทริบิวต์ และ เมธอด

- **คลาส (Class)** คือ เป็นต้นแบบไว้สร้างออบเจกต์ที่มีลักษณะเดียวกันหรือเพื่อเก็บออบเจกต์ต่างๆ
- **ออบเจกต์ (Object)** คือ สิ่งต่าง ๆ รอบตัว ซึ่งมีคุณลักษณะ (Attribute) และความสามารถในการทำงาน (Method)
- **แอตทริบิวต์ (Attribute)** คือ คุณสมบัติที่ใช้นิยามความหมายของออบเจกต์
- **เมธอด (Method)** คือ พฤติกรรมที่ออบเจกต์แต่ละตัวสามารถทำได้



# คลาส ออบเจกต์ แอตทริบิวต์ และ เมธอด

## ตัวอย่างของ ออบเจกต์ - แอตทริบิวต์

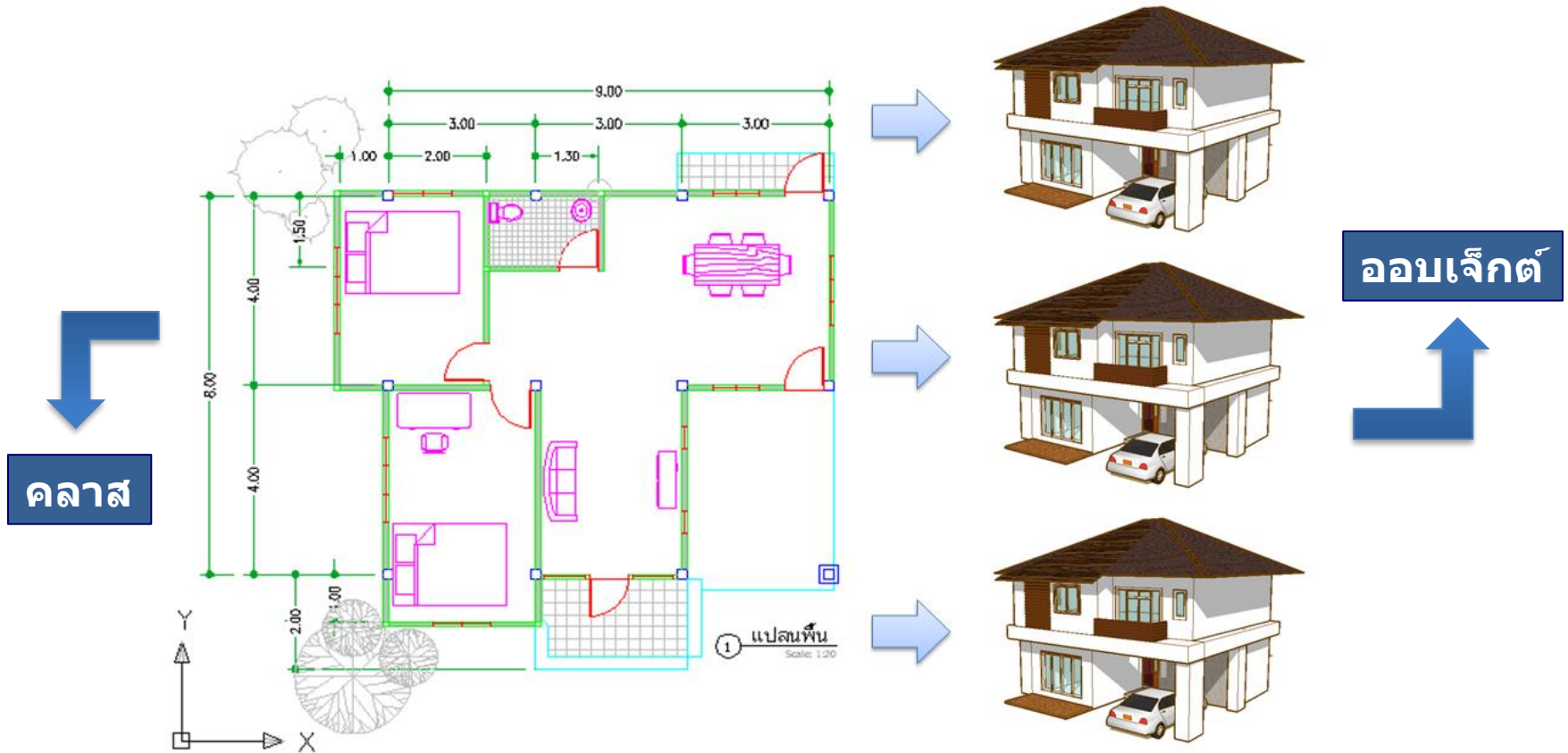
- **ออบเจกต์** คน จะมี **แอตทริบิวต์** คือ หมายเลขประจำตัวประชาชน, วันเกิด, เพศ, เบอร์โทรศัพท์ ฯลฯ
- **ออบเจกต์** สุนัข จะมี **แอตทริบิวต์** คือ ชื่อ, สี, พันธุ์ ฯลฯ
- **ออบเจกต์** จักรยาน จะมี **แอตทริบิวต์** คือ ???

## ตัวอย่างของ ออบเจกต์ - เมธอด

- **ออบเจกต์** คน จะมี **เมธอด** คือ เดิน, ยืน, นอน, มีบัตร ฯลฯ
- **ออบเจกต์** สุนัข จะมี **เมธอด** คือ ???
- **ออบเจกต์** จักรยาน จะมี **เมธอด** คือ ???



# คลาส ออกแบบjekt แอตทริบิวต์ และ เมธอด



# ขั้นตอนการใช้งานคลาส

---

- การประกาศคลาส
- การประกาศแอตทริบิวต์
- การประกาศเมธอด
- การประกาศออบเจกต์
- การเข้าถึงสมาชิกของคลาส





# การประกาศคลาส

---

```
[modifier] class ClassName {  
    [AttributeName]  
    [MethodName]  
}
```

โดยที่

|               |                                               |
|---------------|-----------------------------------------------|
| modifier      | เป็นคีย์เวิร์ดที่กำหนดคุณสมบัติการเข้าถึงคลาส |
| ClassName     | เป็นชื่อคลาส                                  |
| AttributeName | เป็นส่วนของการประกาศแอตทริบิวต์               |
| MethodName    | เป็นส่วนของการประกาศเมธอด                     |



# การประกาศแอตทริบิวต์

---

[**modifier**] **dataType** **AttributeName**;

โดยที่

|                      |                                                      |
|----------------------|------------------------------------------------------|
| <b>modifier</b>      | เป็นคีย์เวิร์ดที่กำหนดคุณสมบัติการเข้าถึงแอตทริบิวต์ |
| <b>dataType</b>      | เป็นชนิดข้อมูลของแอตทริบิวต์                         |
| <b>AttributeName</b> | เป็นชื่อแอตทริบิวต์                                  |



# การประกาศเมธอด

```
[modifier] return_type Methodname ([parameter])  
{  
    [method_body]  
    return varValue;  
}
```

โดยที่

modifier

return\_type

MethodName

parameter

method\_body

varValue

เป็นคีย์เวิร์ดที่กำหนดคุณสมบัติการเข้าถึงเมธอด

เป็นชนิดของข้อมูลที่เมธอดจะส่งค่ากลับ

ในกรณีที่ไม่มีค่าส่งกลับ ให้กำหนดเป็น **void**

เป็นชื่อเมธอด

เป็นตัวแปรที่ใช้ในการรับข้อมูล

เป็นชุดคำสั่งการทำงานของเมธอด

เป็นค่าที่ต้องการส่งค่ากลับ ในกรณีที่กำหนดให้  
return\_type เป็น void จะไม่มีคำสั่ง return

# การประกาศออบเจกต์

---

สามารถสร้างออบเจกต์ได้ตามรูปแบบดังนี้

```
ClassName ObjectName;  
ObjectName = new ClassName();
```

หรือ

```
ClassName ObjectName = new ClassName();
```

โดยที่

|                   |                                        |
|-------------------|----------------------------------------|
| ClassName         | เป็นชื่อของคลาสที่ใช้สร้างออบเจกต์นั้น |
| <b>ObjectName</b> | เป็นชื่อออบเจกต์ที่ประกาศใช้งาน        |



# การเข้าถึงสมาชิกของคลาส

---

## ตัวแปร

ObjectName.**AttributeName**;

โดยที่

|                      |                                     |
|----------------------|-------------------------------------|
| ObjectName           | เป็นชื่อออบเจกต์                    |
| <b>AttributeName</b> | เป็นชื่อแอตทริบิวต์ที่ต้องการใช้งาน |

## เมธอด

ObjectName.**MethodName** ([**argument**]);

หรือ

dataType **MethodValue** = ObjectName.**MethodName**([**argument**]);

โดยที่

|                    |                                                    |
|--------------------|----------------------------------------------------|
| ObjectName         | เป็นชื่อออบเจกต์                                   |
| <b>MethodName</b>  | เป็นชื่อเมธอดที่ต้องการใช้งาน                      |
| <b>argument</b>    | เป็นค่าที่ต้องการส่งผ่านไปให้เมธอดที่ต้องการใช้งาน |
| dataType           | เป็นชนิดข้อมูลของเมธอดที่มีการคืนค่า               |
| <b>MethodValue</b> | เป็นตัวแปรที่ใช้เก็บค่าที่ได้จากการคืนค่าของเมธอด  |

# ตัวอย่างการสร้างออบเจกต์และการใช้คลาส

```
public class Box {  
    // constant as required by the problem  
    public static final double WT_PER_SQR_IN = 25.0;  
    // data fields as specified in the class notation  
    private double width;  
    private double height;  
    private double depth;
```

← ชื่อคลาส

} ตัวแปรและ  
ค่าคงที่ของคลาส



# ตัวอย่างการสร้างออบเจกต์และการใช้คลาส

```
// constructor
```

```
public Box(double w, double h, double d) {  
    width = w;    height = h;    depth = d;  
}  
  
public double getWidth() {  
    return width;  
}  
  
public double getHeight() {  
    return height;  
}  
  
public double getDepth() {  
    return depth;  
}
```

← คอนสตรัคเตอร์

เมธอด



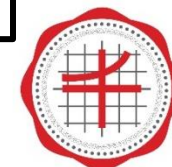
Faculty of Engineering  
SRINAKHARINWIROT UNIVERSITY

# ตัวอย่างการสร้างออบเจกต์และการใช้คลาส

```
public class TestBox {  
    public static void main(String[] args) {  
        Box b1 = new Box(2, 3, 4);  
        Box b2 = new Box(4, 5, 3);  
        Box b3 = new Box(10, 10, 10);  
        printBox("b1", b1);  
        printBox("b2", b2);  
        printBox("b3", b3);  
        // Check canContain()  
        System.out.println("Verifying canContain()...");  
        System.out.println("b2 can contain b1? " + b2.canContain(b1));  
        System.out.println("b2 can contain b3? " + b2.canContain(b3));  
    }  
}
```



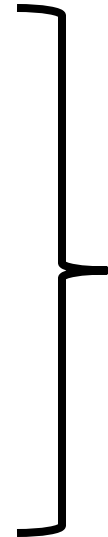
วิธีการในการสร้างออบเจกต์





# ตัวอย่างการสร้างออบเจกต์และการใช้คลาส

```
private static void printBox(String name, Box b) {  
    System.out.println("Properties of Box " + name);  
    System.out.println("(w, h, d) = " + "(" + b.getWidth() + ", " +  
b.getHeight() + ", " + b.getDepth() + ") in cm");  
    System.out.println("Volume in cm^3 = " + b.getVolume());  
    System.out.println("Surface area in cm^2 = " + b.getSurfaceAreaAllSides());  
    System.out.println("Weight in grams = " + b.getWeight());  
    System.out.println();  
}  
}
```



การเรียกใช้  
สมาชิกในคลาส

# Modifier ในภาษา Java

▪ **Non Access modifier** คือคีย์เวิร์ดที่ใช้กำหนดคุณสมบัติอื่นๆ ที่ไม่ใช่ระดับการเข้าถึงข้อมูลของคลาสหนึ่ง ๆ เช่น

- **static** ใช้เป็นคีย์เวิร์ดสำหรับกำหนดให้แอตทริบิวต์หรือเมธอดใดๆ

**แอตทริบิวต์ที่เป็น static ควรมีคุณสมบัติดังนี้**

- แอตทริบิวต์นั้นถูกสร้างขึ้นครั้งแรกเพียงครั้งเดียว
- แอตทริบิวต์นั้นถูกเรียกใช้บ่อย ๆ และถูกเรียกใช้จากหลายออบเจกต์

**เมธอดที่เป็นแบบ static จะมีคุณสมบัติดังนี้**

- เมธอดนั้นถูกเรียกใช้งานได้ตลอดเวลาที่อยู่ภายในขอบเขตของคลาส
- เมธอดนั้นถูกเรียกใช้จากคลาสอื่นผ่านชื่อคลาสได้ โดยไม่ต้องใช้ออบเจกต์

- **final** ใช้เป็นคีย์เวิร์ดสำหรับกำหนดให้แอตทริบิวต์หรือคลาสใดๆ

- แอตทริบิวต์นั้นจะไม่สามารถถูกเปลี่ยนแปลงค่าได้ ตลอดการทำงานของโปรแกรม
- คลาสนั้นจะไม่อนุญาตให้คลาสอื่นมาสืบทอดคุณสมบัติได้

# Modifier ในภาษา Java

▪ **Access modifier** คือ คีย์เวิร์ดที่ใช้กำหนดระดับการเข้าถึงข้อมูล เพื่อป้องกันการเปลี่ยนแปลงข้อมูลภายในคลาส

ระดับของ **access modifier** มี 4 ระดับ คือ

- **public** สามารถเข้าใช้งานได้จากทุกคลาส
- **private** สามารถเข้าใช้งานข้อมูลภายในคลาสเดียวกันเท่านั้น
- **protected** สามารถเข้าใช้งานข้อมูลภายในคลาสเดียวกันและคลาสที่สืบทอดกัน
- **package** สามารถเข้าใช้งานข้อมูลภายในคลาสเดียวกันและคลาสอื่นที่อยู่ภายในแพ็คเกจเดียวกัน



# การกำหนดระดับของ **access modifier**

- กรณีที่ข้อมูลทำหน้าที่เป็นตัวแปรสาธารณะอนุญาตให้คลาสใด ๆ เรียกใช้ได้โดยตรง ให้กำหนดเป็น **public**
- กรณีที่ข้อมูลมีค่าเป็นส่วนตัวไม่อนุญาตให้คลาสอื่นมาเปลี่ยนแปลงค่าได้ ให้กำหนดเป็น **private**
- กรณีที่ข้อมูลมีความสัมพันธ์ในลักษณะที่มีการสืบทอดจากคลาสแม่สู่คลาสลูก ให้กำหนดเป็น **protected**
- กรณีที่เป็นข้อมูลทั่วไป การไม่กำหนดระดับของ access modifier จะถือว่าเป็นระดับการใช้งานเป็น **package** ซึ่งมีการป้องกันระดับการใช้งานดังนี้
  - ข้อมูลสามารถถูกเรียกใช้จากคลาสที่อยู่ภายในแพ็คเกจเดียวกัน
  - ถ้าต้องการใช้ข้อมูลจากคลาสต่างแพ็คเกจกัน จะต้องกำหนดเป็นแบบ public
  - ใช้ในกรณีที่ไม่ต้องการให้ตัวแปรเป็นสาธารณะและไม่ต้องการให้เป็นตัวแปรที่มีค่าส่วนตัว
  - สามารถจำกัดสิทธิการใช้งานภายในแพ็คเกจเดียวกัน

# ระดับการใช้งานแบบ **public**

- เป็นการกำหนดแอตทริบิวต์หรือเมธอดที่ต้องการ ให้สามารถเรียกใช้ได้จากคลาสใด ๆ

```
public class public_Ac_Mod{  
    public static void main(String[] args) {  
        public_emp emp = new public_emp();  
        System.out.println("Current Salary is = " + emp.salary);  
        emp.salary = 24000.0f;  
        System.out.println("New Salary is = " + emp.salary);  
    }  
}
```

```
public class public_emp {  
    public float salary = 15000.0f;  
}
```

> run public\_Ac\_Mod

Current Salary is = 15000.0

New Salary is = 24000.0



# ระดับการใช้งานแบบ **private**

- เป็นการกำหนดแอตทริบิวต์หรือเมธอดที่ต้องการ ให้สามารถเรียกใช้ได้จากคลาสเดียวกันเท่านั้น ไม่อนุญาตให้คลาสอื่นเรียกใช้งาน

```
public class private_Ac_Mod {  
    public static void main(String[] args) {  
        private_emp emp = new private_emp();  
        System.out.println("Current Salary is = " + emp.getsalary());  
        System.out.println("New Salary is = " + emp.setsalary(23000));  
    }  
}
```

> run private\_Ac\_Mod

Current Salary is = 15000.0

New Salary is = 23000.0

```
public class private_emp {  
    private float salary = 15000.0f;  
    public float getsalary() {  
        return salary;  
    }  
}
```

```
public float setsalary(float s) {  
    salary = s;  
    return salary;  
}
```



# ระดับการใช้งานแบบ **protected**

- เป็นการกำหนดแอตทริบิวต์หรือเมธอดที่อนุญาตให้คลาสที่มีความสัมพันธ์จากคลาสแม่สู่คลาสลูกใช้งานเท่านั้น ไม่อนุญาตให้คลาสอื่นเรียกใช้ได้

```
class protected_Ac_Mod {  
    public static void main(String[] args) {  
        Ac_Mod emp = new Ac_Mod();  
        emp.setsalary();  
    }  
}  
class Ac_Mod extends protected_emp {  
    void setsalary() {  
        System.out.println("Current Salary is = " + salary);  
        salary = 22000.0f;  
        System.out.println("New Salary is = " + salary);  
    }  
}
```

```
public class protected_emp {  
    protected float salary = 15000.0f;  
}
```

> **run protected\_Ac\_Mod**

Current Salary is = 15000.0

New Salary is = 22000.0



# ระดับการใช้งานแบบ **package**

- เป็นการกำหนดแอตทริบิวต์หรือเมธอดที่ต้องการใช้งานในคลาสเดียวกันหรือคลาสที่อยู่ในแพ็คเกจเดียวกัน โดยจะไม่สามารถทำการเรียกใช้จากคลาสที่อยู่ต่างแพ็คเกจกันได้ คลาสที่อยู่ในแพ็คเกจเดียวกันหมายถึงคลาสที่อยู่ในโพลเดอร์เดียวกัน

```
public class package_Ac_Mod {  
    public static void main(String[] args) {  
        package_emp emp = new package_emp();  
        System.out.println("Current Salary is = " + emp.salary);  
        emp.salary = 21000.0f;  
        System.out.println("New Salary is = " + emp.salary);  
    }  
}
```

```
public class package_emp {  
    double salary = 15000.0f;  
}
```

```
> run package_Ac_Mod  
Current Salary is = 15000.0  
New Salary is = 21000.0
```





# แพ็คแกลและการใช้งาน

- แพ็คแกลเป็นการจัดหมวดหมู่ของคลาสเพื่อให้เป็นระเบียบและง่ายต่อการเรียกใช้งาน
- เปรียบได้กับโฟลเดอร์ที่มีไว้รวบรวมคลาสให้อยู่ด้วยกัน โดยใช้การตั้งชื่อแบบโครงสร้างต้นไม้ ซึ่งมีเครื่องหมาย "." คั่น
- ตัวอย่าง เช่น

**Tax\_Calculation.tax** หมายถึง คลาส tax อยู่ในแพ็คเกจ Tax\_Calculation



# ตัวอย่างโปรแกรมที่มีการใช้แพ็คเกจ

- การอ้างอิงคลาสที่อยู่ในแพ็คเกจเดียวกัน

```
package Tax_Calculation;  
  
public class tax {  
    public float tax_rate = 0.10f;  
}
```

```
package Tax_Calculation;  
  
public class vat1 {  
    public static void main(String[] args) {  
        tax x = new tax();  
        System.out.println("tax_rate = " + x.tax_rate);  
    }  
}
```

```
> run Tax_Calculation.vat1  
tax_rate = 0.1
```



# ตัวอย่างโปรแกรมที่มีการใช้แพ็คเกจ

- การอ้างอิงคลาสจากแพ็คเกจอื่น

```
package Tax_Calculation;  
  
public class tax {  
    public float tax_rate = 0.10f;  
}
```

```
> run Vat_Calculation.vat2  
tax_rate = 0.1
```

```
package Vat_Calculation;  
import Tax_Calculation.tax;  
  
public class vat2 {  
    public static void main(String[] args) {  
        tax x = new tax();  
        System.out.println("tax_rate = " + x. tax_rate);  
    }  
}
```



# แพ็คเกจพื้นฐานของภาษาจาวา

---

- **java.util** เป็นคลาสที่ใช้จัดการเกี่ยวกับวันเวลาและคลาสอรรถประโยชน์อื่นๆ
- **java.applet** เป็นคลาสสำหรับใช้สร้างจาวาแอปเพลต
- **java.awt** และ **javax.swing** เป็นคลาสที่ใช้สร้างส่วนติดต่อกับผู้ใช้แบบกราฟิก



# CPE 121 Mobile Application Developments

---

## การแสดงผลและการรับข้อมูล (Data Output and Input)



**Faculty of Engineering**

SRINAKHARINWIROT UNIVERSITY

---

**Pramual Choorat, Ph.D.**

**Department of Computer Engineering**

# การแสดงผลข้อมูลด้วย **println ()** หรือ **print()**

## รูปแบบการใช้งาน

**System.out.println(argument<sub>1</sub> + argument<sub>2</sub> + ... + argument<sub>n</sub>)**

หรือ

**System.out.print(argument<sub>1</sub> + argument<sub>2</sub> + ... + argument<sub>n</sub>)**

โดยที่

argument<sub>1</sub>, argument<sub>2</sub>, argument<sub>n</sub> เป็นข้อมูลที่ต้องการแสดงผลอาจจะเป็นข้อความ ตัวแปร หรือนิพจน์ ซึ่งสามารถเขียนต่อกันได้โดยใช้เครื่องหมาย "+"



# รหัสควบคุมการแสดงผล

| อักขระพิเศษ  | ความหมาย                                                   |
|--------------|------------------------------------------------------------|
| <b>\b</b>    | เลื่อนเคอร์เซอร์ถอยหลังไป 1 ตัวอักษร                       |
| <b>\f</b>    | ขึ้นหน้าใหม่                                               |
| <b>\n</b>    | ขึ้นบรรทัดใหม่                                             |
| <b>\r</b>    | เลื่อนเคอร์เซอร์ไปทางซ้ายสุด                               |
| <b>\t</b>    | แสดงแท็บตามแนวนอน                                          |
| <b>\'</b>    | แสดงเครื่องหมาย '                                          |
| <b>\"</b>    | แสดงเครื่องหมาย "                                          |
| <b>\\</b>    | แสดงเครื่องหมาย \                                          |
| <b>\xxx</b>  | แสดงตัวอักษรรหัส Ascii เช่น \065 เป็นการแสดงตัวอักษร A     |
| <b>\uxxx</b> | แสดงตัวอักษรรหัส Unicode เช่น \u0008 เป็นการขึ้นบรรทัดใหม่ |



# ข้อแตกต่างระหว่างเมธอด **println()** และ **print()**

- เมธอด **println()** เมื่อแสดงผลแล้วตัวชี้ตำแหน่งจะอยู่ตำแหน่งต้นของบรรทัดใหม่
- เมธอด **print()** เมื่อแสดงผลแล้วตัวชี้ตำแหน่งจะอยู่ตำแหน่งสุดท้ายของข้อมูลที่แสดงผล ดังตัวอย่างต่อไปนี้

```
public class Print_Test {  
    public static void main(String[] args) {  
        System.out.print("Good Morning!!! ");  
        System.out.print("How are you ?\n");  
        System.out.println("I am JAVA");  
        System.out.println("Are you JAVA ?");  
    }  
}
```

> run Print\_Test

Good Morning!!! How are you ?

I am JAVA

Are you JAVA ?





# การแสดงผลข้อมูลด้วยเมธอด **printf()**

## รูปแบบการใช้งาน

**System.out.printf(Control\_String, argument<sub>1</sub>, argument<sub>2</sub>,  
...,argument<sub>n</sub>)**

โดยที่

**Control\_String** ประกอบด้วย รหัสควบคุมการแสดงผล, รหัสการแสดงผลและ  
ส่วนขยายรหัสการแสดงผล argument<sub>1</sub>, argument<sub>2</sub>, argument<sub>n</sub> เป็นข้อมูลที่  
ต้องการแสดงผล



# รหัสการแสดงผลและส่วนขยายรหัสการแสดงผล

| รหัสการแสดงผล | ชนิดของข้อมูลที่ใช้           |
|---------------|-------------------------------|
| %c            | ตัวอักษร                      |
| %d            | เลขจำนวนเต็ม                  |
| %f            | เลขทศนิยม                     |
| %e            | เลขในรูป exponential          |
| %s            | ข้อความ                       |
| %u            | เลขจำนวนเต็มไม่คิดเครื่องหมาย |

| ส่วนขยาย      | รายละเอียดการใช้งาน                                                                                                                                        |
|---------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| -             | ให้แสดงผลจากซ้ายไปขวา                                                                                                                                      |
| +             | ให้แสดงผลจากขวามาซ้าย                                                                                                                                      |
| ตัวเลข        | กำหนดจำนวนตำแหน่งสำหรับการแสดงผล<br>ถ้าเป็นการแสดงผลจากขวามาซ้าย และจำนวน<br>ข้อมูลที่แสดงผลน้อยกว่าจำนวนตำแหน่งจะเติม<br>ส่วนที่เหลือด้านซ้ายด้วยช่องว่าง |
| . (จุดทศนิยม) | กำหนดรูปแบบการแสดงผลให้มีจุดทศนิยม                                                                                                                         |

# ตัวอย่างการใช้งานเมธอด **printf()**

```
public class Printf_Test {  
    public static void main(String[] args){  
        int i = 3277;  
        double d = 132324.25;  
        System.out.printf("%-10s%5d\n","%5d",i);  
        System.out.printf("%-10s%f\n", "%f",d);  
        System.out.printf("%-10s%16f\n","%16f",d);  
        System.out.printf("%-10s%16.2f\n","%16.2f", d);  
        System.out.printf("%-10s%,16.2f\n","%,16.2f",d);  
    }  
}
```

> run Printf\_Test

|         |               |
|---------|---------------|
| %5d     | 3277          |
| %f      | 132324.250000 |
| %16f    | 132324.250000 |
| %16.2f  | 132324.25     |
| %,16.2f | 132,324.25    |



# การแสดงผลข้อมูลด้วยคลาส **JOptionPane**

- เมธอด **showMessageDialog()** มีรูปแบบการใช้งาน ดังนี้

`JOptionPane.showMessageDialog(Parent_Window, Message, Title, Type);`

โดยที่

|               |                                                                                                     |
|---------------|-----------------------------------------------------------------------------------------------------|
| Parent_Window | เป็นชื่อของ parent window ที่ต้องการแสดงผล<br>ถ้าค่าเป็น <b>null</b> จะแสดงไดอะล็อกบ็อกซ์กลางหน้าจอ |
| Message       | เป็นข้อความที่ต้องการแสดงในไดอะล็อกบ็อกซ์                                                           |
| Title         | เป็นข้อความที่ปรากฏในส่วนของ Title bar ของ<br>ไดอะล็อกบ็อกซ์                                        |
| Type          | เป็นชนิดของไดอะล็อกบ็อกซ์ ซึ่งจะถูกกำหนดโดยค่าคงที่<br>ต่อไปนี้                                     |



# การแสดงผลข้อมูลด้วยคลาส **JOptionPane**

- **Type** เป็นชนิดของไดอะล็อกบ็อกซ์ ซึ่งจะถูกกำหนดโดยค่าคงที่ต่อไปนี้
  - ERROR\_MESSAGE** : เป็นการแสดงข้อผิดพลาด และแสดงสัญลักษณ์ 
  - INFORMATION\_MESSAGE** : เป็นการแสดงข้อความทั่วไป และแสดงสัญลักษณ์ 
  - PLAIN\_MESSAGE** : เป็นการแสดงข้อความทั่วไป โดยไม่มีการแสดงสัญลักษณ์
  - QUESTION\_MESSAGE** : เป็นการแสดงในลักษณะคำถาม และแสดงสัญลักษณ์ 
  - WARNING\_MESSAGE** : เป็นการแสดงในลักษณะแจ้งเตือน และแสดงสัญลักษณ์ 
- ในกรณีที่ไม่มีกำหนดค่าในส่วนของ **Title** และ **Type**
  - Title** จะถูกกำหนดเป็น **"Message"**
  - Type** จะถูกกำหนดเป็น **INFORMATION\_MESSAGE**

```
import javax.swing.JOptionPane;

public class JOptionPane_Display_Test {

    public static void main(String[] args) {

        JOptionPane.showMessageDialog(null, "My Name is JAVA");

    }

}
```

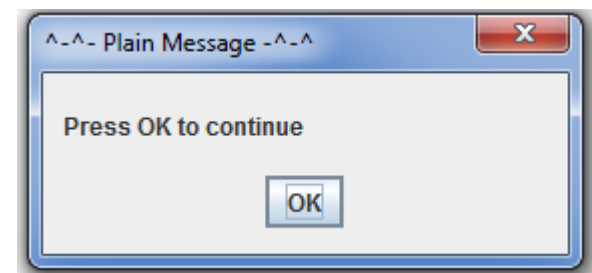
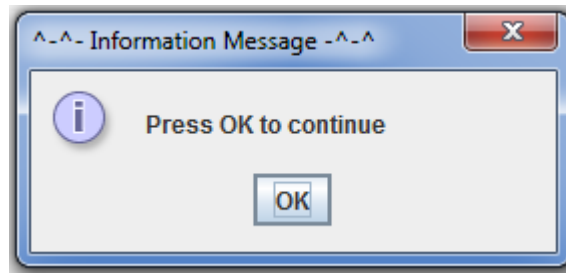


# ตัวอย่าง การใช้เมธอด **showMessageDialog()**

```
JOptionPane.showMessageDialog(null, "Divided By Zero", "^-^- Error Message ^-^", JOptionPane.ERROR_MESSAGE);
```

```
JOptionPane.showMessageDialog(null, "Press OK to continue", "^-^- Information Message ^-^", JOptionPane.INFORMATION_MESSAGE);
```

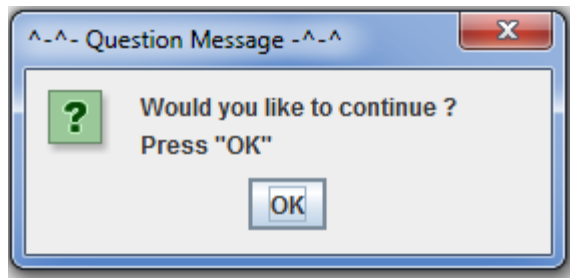
```
JOptionPane.showMessageDialog(null, "Press OK to continue", "^-^- Plain Message ^-^", JOptionPane.PLAIN_MESSAGE);
```



# ตัวอย่าง การใช้เมธอด **showMessageDialog()**

```
JOptionPane.showMessageDialog(null, "Would you like to continue ? \n" + "Press  
\"OK\"", "^-^- Question Message -^-^ ", JOptionPane.QUESTION_MESSAGE);
```

```
JOptionPane.showMessageDialog(null, "Variable is not used", "^-^- Warning Message -^-  
^ ", JOptionPane.WARNING_MESSAGE);
```



# การแสดงผลด้วยคลาส **DecimalFormat**

- เมธอด **format()** มีรูปแบบการใช้งาน ดังนี้

```
DecimalFormat df= new DecimalFormat(argument);  
String str= df.format(payment);
```

โดยที่

**argument** เป็นรูปแบบการแสดงผลที่ต้องการ ประกอบด้วย

- 0 แทนตัวเลข 0 ในกรณีที่ต้องการให้แสดงตัวเลข
- "#" แทนตัวเลขใดๆ ที่ไม่ใช่ 0 ในกรณีที่เป็เลข 0 จะไม่แสดงผล
- "," ให้แสดงผลโดยมีเครื่องหมาย "," เป็นตัวคั่นในหลักพัน

df เป็นชื่อออบเจกต์ของคลาสที่สร้างขึ้น

str เป็นชื่อตัวแปรที่ใช้รับค่าที่กำหนดรูปแบบ





# ตัวอย่างการใช้งานเมธอด **format()**

```
import java.text.DecimalFormat;

public class DecimalFormat_Test {

    public static void main(String[] args){
        double payment;      payment = 12345.00;
        System.out.println("payment = 12345.00");
        DecimalFormat nf = new DecimalFormat("#,###.00");
        System.out.println("#,###.00 = "+ nf.format(payment));
        nf = new DecimalFormat("#####.##");
        System.out.println("#,###.## = "+ nf.format(payment));
        payment = 0.5111;      System.out.println("payment = 0.5111");
        nf = new DecimalFormat("#.0000");
        System.out.println("#.0000 = "+ nf.format(payment));
        nf = new DecimalFormat("0.0000");
        System.out.println("0.0000 = "+ nf.format(payment));  }}
```

> run DecimalFormat\_Test

payment = 12345.00

#,###.00 = 12,345.00

#,###.## = 12345

payment = 0.5111

#.0000 = .5111

0.0000 = 0.5111

# การรับข้อมูลด้วยคลาส **InputStreamReader**

- ทำงานร่วมกับคลาส **BufferedReader** รับข้อมูลครั้งละ 1 บรรทัด
- ใช้เมธอด **readLine()** ซึ่งเป็นการรับข้อมูลชนิดข้อความ (**String**)
- จะต้อง **import** แพคเกจ **java.io** ก่อนเสมอ มีรูปแบบการใช้งานดังนี้

```
InputStreamReader reader = new InputStreamReader(System.in);  
BufferedReader stdin = new BufferedReader(reader);
```

หรือ

```
BufferedReader stdin = new BufferedReader(new  
    InputStreamReader(System.in));
```

โดยที่

reader เป็นชื่อออบเจกต์ของคลาส **InputStreamReader**  
stdin เป็นชื่อออบเจกต์ของคลาส **BufferedReader**



# ตัวอย่างการรับข้อมูลด้วย **InputStreamReader**

```
import java.io.*;

public class InputStream_Test_1 {

    public static void main(String[] args) throws IOException {

        String input;

        BufferedReader stdin = new BufferedReader(new InputStreamReader(System.in));

        System.out.print("ป้อนข้อความที่นี่ >>> ");

        input = stdin.readLine();

        System.out.println("ข้อความของท่านคือ "+input);

    }

}
```

> run **InputStream\_Test\_1**

ป้อนข้อความที่นี่ >>> **Java Programming**

ข้อความของท่านคือ Java Programming

# ตัวอย่างการรับข้อมูลด้วย **InputStreamReader**

```
import java.io.*;

public class InputStream_Test_3 {

    public static void main(String[] args) throws IOException {

        String input;

        BufferedReader stdin = new BufferedReader(new InputStreamReader(System.in));

        System.out.print("ป้อนข้อมูลเงินบาทที่นี่ >>> ");

        input = stdin.readLine();    double money = Double.parseDouble(input);

        System.out.print("ป้อนข้อมูลอัตราแลกเปลี่ยนเงินบาทต่อ 1 ดอลลาร์ ที่นี่ >>> ");

        input = stdin.readLine();    double dollar_rate = Double.parseDouble(input);

        System.out.println("แลกเงินดอลลาร์ได้ " + (money / dollar_rate)+ " ดอลลาร์");

    }

}
```

**> run InputStream\_Test\_3**

ป้อนข้อมูลเงินบาทที่นี่ >>> **100**

ป้อนข้อมูลอัตราแลกเปลี่ยนเงินบาทต่อ 1 ดอลลาร์ ที่นี่ >>> **32**

แลกเงินดอลลาร์ได้ 3.125 ดอลลาร์

# การรับข้อมูลชนิดข้อความด้วยคลาส **Scanner**

- รับข้อมูลชนิดของข้อมูลเป็นข้อความหรือตัวเลขก็ได้
- ต่างจากการรับข้อมูลด้วยคลาส `InputStreamReader` ที่รับข้อมูลได้เฉพาะข้อความเท่านั้น
- การใช้งานคลาส `Scanner` ต้องสร้างออบเจกต์จากคลาส `Scanner` ก่อน
- จะต้อง `import` แพคเกจ **`java.util.Scanner`** และประกาศออบเจกต์ตามรูปแบบ ดังนี้

```
Scanner sn = new Scanner(System.in);
```

โดยที่

|                        |                                                                     |
|------------------------|---------------------------------------------------------------------|
| <code>sn</code>        | เป็นชื่อออบเจกต์ที่สร้างจากคลาส <b>Scanner</b>                      |
| <code>System.in</code> | เป็นช่องทางการนำเข้าข้อมูลมาตรฐาน หมายถึง การอ่านข้อมูลจากแป้นพิมพ์ |

# เมธอดของคลาส **Scanner**

---

- **nextInt()** สำหรับรับข้อมูลประเภทเลขจำนวนเต็ม
- **nextFloat()** สำหรับรับข้อมูลประเภทเลขจำนวนทศนิยมชนิด **float**
- **nextDouble()** สำหรับรับข้อมูลประเภทเลขจำนวนทศนิยมชนิด **double**
- **nextLine()** สำหรับรับข้อมูลประเภทข้อความ



# เมธอดของคลาส Scanner

```
import java.util.Scanner;

public class Scanner_Test {

    public static void main(String [] args) {
        String name; int Qty; double price;
        Scanner sn = new Scanner(System.in);
        System.out.print("ป้อนข้อมูลชื่อสินค้า >>> ");
        name = sn.nextLine();
        System.out.print("ป้อนข้อมูลจำนวนสินค้า(หน่วย) >>> ");
        Qty = sn.nextInt();
        System.out.print("ป้อนข้อมูลราคาสินค้าต่อหน่วย(บาท) >>> ");
        price = sn.nextDouble();
        System.out.printf("%s %d หน่วย ",name,Qty);
        System.out.printf("หน่วยละ %.2f บาท\n",price);
        System.out.printf("รวมเป็นเงิน %.2f บาท\n" , Qty * price);
        System.out.printf("ภาษี 7%% เป็นเงินสุทธิ %.2f บาท\n", 1.07f*(Qty * price));
    }
}
```

## > run Scanner\_Test

```
ป้อนข้อมูลชื่อสินค้า >>> mama
ป้อนข้อมูลจำนวนสินค้า(หน่วย) >>> 6
ป้อนข้อมูลราคาสินค้าต่อหน่วย(บาท) >>> 6
mama 6 หน่วย หน่วยละ 6.00 บาท
รวมเป็นเงิน 36.00 บาท
ภาษี 7% เป็นเงินสุทธิ 38.52 บาท
```



# การรับผลข้อมูลด้วยคลาส **JOptionPane**

- เมธอด **showInputDialog()** มีรูปแบบการใช้งาน ดังนี้

```
String data = JOptionPane.showInputDialog(Parent_Window,  
                                           Message, Title, Type);
```

โดยที่

**Parent\_Window**

เป็นชื่อของ parent window ที่ต้องการแสดงผล  
ถ้าค่าเป็น null จะแสดงไดอะล็อกบ็อกซ์กลางหน้าจอ

**Message**

เป็นข้อความที่ต้องการแสดงในไดอะล็อกบ็อกซ์

**Title**

เป็นข้อความที่ปรากฏในส่วนของ Title bar ของ  
ไดอะล็อกบ็อกซ์

**Type**

เป็นชนิดของไดอะล็อกบ็อกซ์ ซึ่งจะถูกกำหนดโดยค่าคงที่  
ต่อไปนี้





# การรับผลข้อมูลด้วยคลาส **JOptionPane**

- **Type** เป็นชนิดของไดอะล็อกบ็อกซ์ ซึ่งจะถูกกำหนดโดยค่าคงที่ต่อไปนี้
  - **ERROR\_MESSAGE** : เป็นการแสดงข้อผิดพลาด และแสดงสัญลักษณ์ 
  - **INFORMATION\_MESSAGE** : เป็นการแสดงข้อความทั่วไป และแสดงสัญลักษณ์ 
  - **PLAIN\_MESSAGE** : เป็นการแสดงข้อความทั่วไป โดยไม่มีการแสดงสัญลักษณ์
  - **QUESTION\_MESSAGE** : เป็นการแสดงในลักษณะคำถาม และแสดงสัญลักษณ์ 
  - **WARNING\_MESSAGE** : เป็นการแสดงในลักษณะแจ้งเตือน และแสดงสัญลักษณ์ 
- ในกรณีที่ไม่มีข้อกำหนดค่าในส่วนของ **Title** และ **Type**
  - Title** จะถูกกำหนดเป็น "Input"
  - Type** จะถูกกำหนดเป็น **QUESTION\_MESSAGE**



# การรับผลข้อมูลด้วยคลาส **JOptionPane**

```
import javax.swing.JOptionPane;

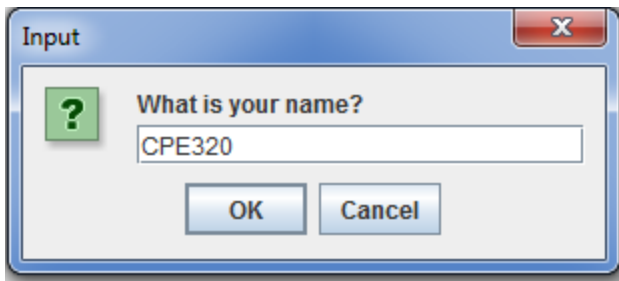
public class JOptionPane_Input_Test {

    public static void main(String[] args) {

        String data = JOptionPane.showInputDialog(null, "What is your name?");
        JOptionPane.showMessageDialog(null, "Your name is " + data);

    }

}
```

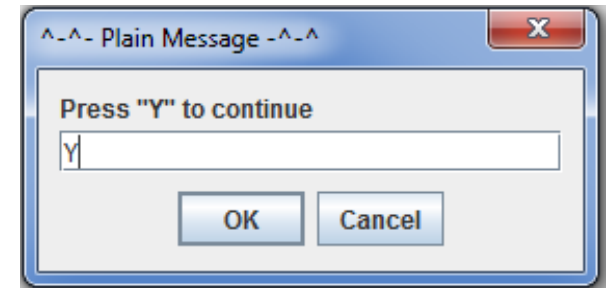
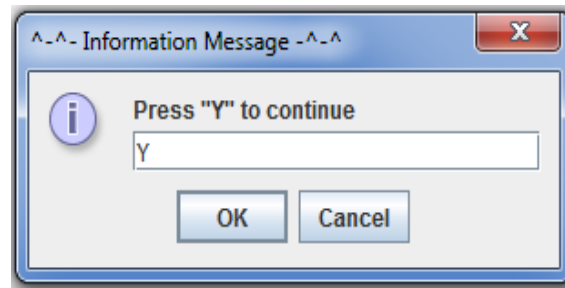
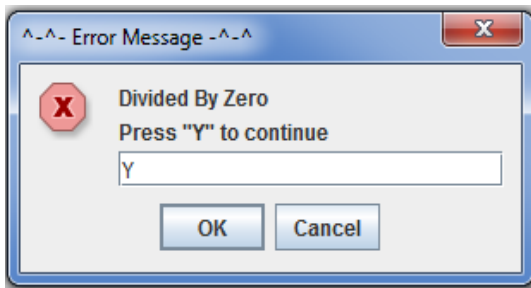


# ตัวอย่าง การใช้เมธอด **showInputDialog()**

```
data = JOptionPane.showInputDialog(null, "Divided By Zero\n"+  
"Press \"Y\" to continue", "^-^ Error Message ^-^",JOptionPane.ERROR_MESSAGE);
```

```
data = JOptionPane.showInputDialog(null, "Press \"Y\" to continue",  
"^-^ Information Message ^-^",JOptionPane.INFORMATION_MESSAGE);
```

```
data = JOptionPane.showInputDialog(null, "Press \"Y\" to continue",  
"^-^ Plain Message ^-^",JOptionPane.PLAIN_MESSAGE);
```



# ตัวอย่าง การใช้เมธอด **showInputDialog()**

```
data = JOptionPane.showInputDialog(null, "Continue ? >> Press \"Y\"",  
"^^- Question Message -^^ ", JOptionPane.QUESTION_MESSAGE);
```

```
data = JOptionPane.showInputDialog(null, "Variable is not used\n"+"Press \"Y\" to  
continue", "^^- Warning Message -^^ ",JOptionPane.WARNING_MESSAGE);
```

